

BEHAVIORAL COLLAPSE IN AI-GENERATED HARNESS POPULATIONS

Anonymous authors

Paper under double-blind review

ABSTRACT

Automatically generated LLM harnesses can differ in source code, executed mechanisms, and correctness outcomes. These differences need not imply reproducible additional capability. We examine the distinction on BIRD text-to-SQL. A discovery population had 36 of 91 harness pairs identical on all evaluated outcomes despite distinct source code; trace inspection found absent mechanisms, untriggered branches, and broken multi-turn paths. A subsequent factorial study crossed strategy forcing and an admission gate over six builders and three seeds, using 1,169 questions and the official BIRD scorer. Free generation already yielded 8.6 pp observed oracle headroom. Post-review reanalysis of the archived experiment estimates the combined intervention at -0.16 pp (95% CI $[-0.83, +0.87]$), with no clear improvement. The implemented gate also excludes two allowed prompt-level strategy types, so this contrast cannot isolate verification alone. Repeated executions show outcome instability, which a single-run oracle can exploit without stable mechanism complementarity. We distinguish observed headroom from stable task-harness advantages and identify the controls needed to connect either to usable selection gains. The current evidence establishes these measurement limits; it does not yet identify stable complementarity beyond repeated sampling.

1 INTRODUCTION

An LLM harness is the program of prompts, tools, and control flow that wraps a frozen model. Automatically generating many such programs creates an evaluation problem: what evidence shows that a new program provides a new capability? Different source code may execute the same path; different paths may produce the same answer; different answers on one run may reflect execution randomness rather than a repeatable advantage.

We study this problem on BIRD text-to-SQL (Li et al., 2023). In a Phase-I discovery population, 21 of 66 generated-candidate pairs had identical correctness vectors. Including the shipped `react` harness and `bare` baseline gives 36 identical pairs out of 91. Pairwise code similarity had no observed rank association with outcome disagreement ($\rho = -0.01$). Trace inspection found three distinct situations: claimed mechanisms absent from control flow, mechanisms present but untriggered on the inspected tasks, and broken multi-turn implementations. These observations motivate an audit of the measurement process; they do not establish that generated harnesses generally collapse.

A subsequent pre-specified factorial study crossed strategy forcing and conformance gating over six builders and three seeds. Its primary evaluation used 1,169 questions from nine databases outside protocol development, with one disclosed 18-item smoke-test exception. Under this revised setting, free ungated generation already produced 8.6 pp observed oracle headroom. The Phase-I pattern therefore did not recur, but changes in generation, judge, data, and execution prevent attribution to any one protocol component without a controlled bridge.

The original gate also changes the admissible strategy space: two allowed single-call prompt transformations cannot pass its mechanism check. Consequently, the intervention combines mechanism screening with verification. We preserve that experiment and report a versioned post-review reanalysis rather than treating it as a repaired-gate trial. The primary contrast remains -0.16 pp (95% CI $[-0.83, +0.87]$). This unresolved estimate neither demonstrates improvement nor establishes equivalence.

	This work	HarnessLens	HarnessBank
Unit of study	population outcome matrix	evolution loop	diversity library
Measures population collapse	yes	no	search collapse only
Gate definition	counterfactual traces	attributable-evidence	semantic screening
Gate calibration	pos/neg controls	—	—
Gate comparison	factorial, mixed screening	gate ablation (single factor)	no
Confirmatory split	9 isolated DBs	—	—

Table 1: Nearest concurrent systems. Behavior-aware verification inside evolution systems is concurrent independent work; our contribution is the population-level outcome measurement, instrument calibration, and the generation-policy contrast. The present gate includes structural screening; the database split has a disclosed 18-item smoke-test exception.

Our contribution at this stage is an empirical account of three measurement boundaries:

1. **Code, trace, and outcome diversity differ.** Discovery examples show why an inactive branch and a missing mechanism require different tests.
2. **Admission and inference define the comparison.** A factorial study and an explicit correction of its data and statistical chain show which intervention was actually evaluated and what its estimate supports.
3. **Observed oracle value is not stable or usable value.** Repeat instability and exploratory selection results motivate separate reporting of oracle accuracy, best-fixed accuracy, repeated-run advantages, and realized selector performance.

The decisive remaining question is whether a real harness population outperforms an equal-budget population of independent executions of the same code, with advantages that survive independent repeats. The current study has not completed that control. We therefore treat stable complementarity as an estimand to be tested, not a discovered property.

2 RELATED WORK

2.1 HARNESS GENERATION FOR LLMs

A growing line of work generates task-specific scaffolding around a frozen model. TTHe evolves text-to-SQL harnesses with an LLM proposer/judge loop (Nie et al., 2026); Self-Harness (Zhang et al., 2026b) shows a model can improve its own harness; JIT-Agent (Zhang et al., 2026a) trains a model to generate a harness per task at inference time; Task-CoEvolve (Miyai et al., 2026) co-evolves tasks and scaffolds. A concurrent 2026 cluster makes harness evolution *behavior-aware from the verification side*: HarnessLens (Xu et al., 2026) allocates rollout budget using behavior-level checks during evolution; HarnessFix (Chen et al., 2026) repairs harnesses from trace-grounded diagnoses; AHE (Lin et al., 2026) verifies each proposed edit against self-declared behavioral predictions; gated semantic quality-diversity search (Luo et al., 2026) evolves populations over LLM-assigned pathology descriptors; Harness Handbook (Wang et al., 2026) attacks behavior localization and outer-loop search.

Our work is complementary but distinct in unit of analysis and question. Table 1 contrasts the two nearest systems. HarnessLens uses behavior as a *signal inside* an evolution loop to allocate compute; we ask whether the resulting *population* is behaviorally diverse as an outcome property, and how it changes under generation-side interventions. HarnessBank (Luo et al., 2026) documents search-space collapse and maintains a semantic diversity library with gated screening; our conformance gate does not score semantics at all — it verifies mechanically, via counterfactual trace probes, that a declared mechanism executes — and our design compares admission policies through a factorial contrast. Because the implemented gate also excludes allowed prompt transformations, the current experiment does not isolate verification’s causal contribution.

2.2 HARNESS CONDITIONAL UTILITY AND ROUTING

Harness quality is instance-conditional: Gupta et al. (2026) and Cho (2026) document that no single harness dominates across instances. HELIX evaluates a full sibling-harness outcome matrix and portfolio complementarity beyond the best fixed candidate, listing a router as future work (Fan & Huang, 2026). STS routes over six fixed human-designed scaffolds (Zhou et al., 2026); GRASP gates skill edits into a development-time library (Moll et al., 2026); Kim (2026) learn instance-level routing over {no-search, search, abstain}; Liu et al. (2026) route models within a harness. Classical algorithm selection (Rice, 1976) and portfolio solvers such as SATzilla (Xu et al., 2008; Kotthoff, 2014) select enumerated algorithms from instance features.

Our candidates are complete automatically generated harnesses. We measure the population outcome structure a selector would inherit (§5.4), rather than build a selector. We diagnose execution paths and compare generation policies through a factorial whose treatment includes structural screening; the present study does not establish stable selectable value.

2.3 BEHAVIORAL DIVERSITY IN LLM POPULATIONS

Diversity of LLM outputs has mostly been studied at the sample level (self-consistency, diverse decoding) or the agent-policy level. The observation that syntactically distinct programs can exhibit identical behavior echoes coverage gaps documented in program synthesis and mutation-testing practice; our contribution is to instantiate and quantify it for AI-generated harness populations, with a mechanism-level taxonomy.

3 PHASE I: MEASURING AND DIAGNOSING THE COLLAPSE (DISCOVERY)

Phase I is a discovery study: its role is to establish the phenomenon, the taxonomy, and the existence of task-side headroom, and to motivate the confirmatory design. None of its results is confirmatory: 24 of its 151 evaluation items overlapped protocol-development measurements, the human control was designed after inspecting development traces, and its judge was a conservative proxy (Appendix A). We report it with those limits stated.

3.1 HARNESSES, OUTCOMES, AND THE THREE METRICS

A **harness** H wraps a frozen target model M : it receives the natural-language task x , issues one or more calls to M , optionally executes intermediate artifacts (e.g. SQL), and returns a final answer scored programmatically. With H_0 the bare single-call baseline and $Y \in \{0, 1\}$ correctness, the **harness marginal value** is $\Delta(x, H, M) = Y(x, H, M) - Y(x, H_0, M) \in \{-1, 0, +1\}$ (+1 repair, -1 harness-induced harm).

We propose that harness-generation and harness-routing pipelines report three precondition metrics on a held-out set, all computable from the outcome matrix any evaluation already produces: **(i) pairwise outcome disagreement** — over all harness pairs, the fraction of tasks where correctness differs; **(ii) union repair rate** — the fraction of bare errors some population member repairs; **(iii) bare-inclusive oracle headroom**

$$H(P) = \text{Acc}(x \mapsto \max_{H \in P \cup \{H_0\}} Y(x, H)) - \max_{H \in P \cup \{H_0\}} \text{Acc}(H).$$

Positive observed headroom records item-dependent successes in that execution matrix; independent repeats are needed to establish a stable routing signal. Disagreement without repair adds no observed coverage beyond bare; repair without headroom adds no observed oracle advantage over the best fixed member.

3.2 THE COLLAPSE, QUANTIFIED

The domain is BIRD text-to-SQL (SQLite; execution-accuracy judging), target frozen at **GLM-5.3-Flash** (temperature 0) throughout Phase I. On the day-1 population — 14 harnesses (bare, react, and 12 evolution candidates) $\times$ 60 tasks — average pairwise outcome disagreement is **3.3%** and **36 of 91 pairs are identical on every task**; among the 12 generated candidates alone (66 candidate

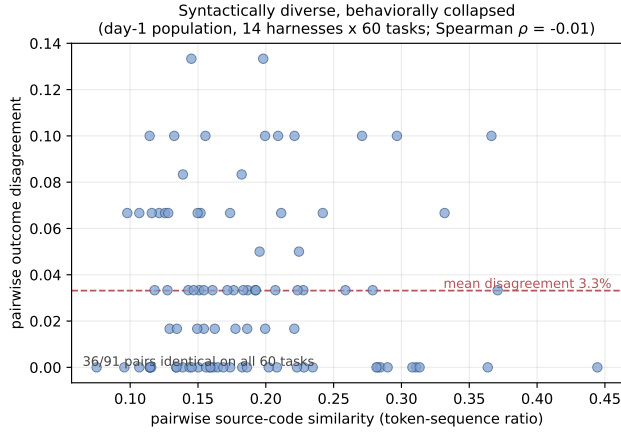


Figure 1: **Syntactic diversity vs. behavioral collapse** (Phase I, discovery; day-1 population: 14 harnesses $\times$ 60 tasks, 91 pairs including the bare and react baselines). Each point is a harness pair; x = token-sequence code similarity, y = fraction of tasks with differing correctness. Code similarity does not predict outcome disagreement ($\rho = -0.01$); 36/91 pairs are identical on all 60 tasks. Raw pairs: `fig1_phenomenon_data.json` in the code release.

pairs; mean pairwise code similarity 0.143 under the normalized-character metric of the original code-difference check), disagreement is 3.8% with 21 identical pairs. Every candidate had passed a code-difference check (“all files distinct, non-trivial diffs”). Union repair over bare errors is 7.7% and oracle headroom 3.3 pp; `react` differs from `bare` on zero tasks.

Figure 1 makes the central relation quantitative: across all 91 pairs, pairwise source-code similarity under the figure’s token-sequence metric (mean 0.190; distinct from the candidates-only normalized-character metric of 0.143 above) has Spearman $\rho = -0.010$ with outcome disagreement — code similarity shows *no observed rank association* with behavioral similarity in this population. (Pairs share harnesses and are not independent samples; we report ρ as a descriptive summary, not an inferential test.)

3.3 TRACE TAXONOMY: WHY CODE DIVERSITY FAILS TO BECOME BEHAVIORAL DIVERSITY

Auditing execution traces (the sequence of LLM calls and artifact executions per task) decomposes “different code” into three failure classes:

- **T1 — mechanism described, not executed (6/12 candidates).** The harness issues exactly one LLM call and never executes an artifact — the identical execution path as `bare`. Docstrings claim retrieval, self-checking, or token caps; none exists in the control flow.
- **T2 — mechanism present but untriggered (`react`, 10/10 audited tasks).** The first SQL executes successfully, so error repair is not triggered and the final SQL equals `bare`’s. This does not show that repair feedback lacks a causal path when a failure occurs.
- **T3 — broken multi-turn path (1/12).** The one candidate implementing genuine multi-turn repair returns prose as final SQL, converting `bare`-correct items into harness-wrong.

The taxonomy is actionable: T1 demands an execution-path acceptance gate, T2 demands controlled faults to test trigger and feedback use, T3 demands executable-output validation — all generation-protocol requirements.

3.4 HUMAN POSITIVE CONTROL: THE HEADROOM EXISTS

To probe whether distinct hand-written mechanisms can produce additional observed coverage, we wrote four harnesses by hand with forcibly distinct execution paths (`hpc_repair`: `execute-and-repair`; `hpc_vote3`: 3-vote on execution results; `hpc_schema`: `schema-linking`; `hpc_hint`: `hint-first reordering`). Same frozen target, tasks, and judge as the collapsed population: oracle headroom

6.67 pp on 60 tasks and **7.28 pp** [3.3, 10.0] on the 151-item set, with repair/schema/hint each producing repairs the others miss. One disclosed caveat: the control was designed *after* inspecting day-1 traces, so its headroom is descriptive, not a blind estimate of stable complementarity. The bare baseline itself flipped 3 of 60 items between the two day-1 runs (API nondeterminism at temperature 0).

3.5 PHASE-I PROTOCOL PROBES AND THEIR LIMITS

Phase I also probed generation-side fixes — a strategy-forced free-form protocol (assigned mechanism, import/smoke acceptance) and a compiled Harness IR. Under the strategy-forced protocol two of three builders produced populations whose point estimates exceeded the pre-specified thresholds (Qwen3.8-Flash 5.96, DeepSeek-V4-Flash-Vision-Exp 7.95 pp; the old-protocol arm missed at 4.64 pp), and per-strategy acceptance ordered with builder identity (GLM-5.3-Flash 3/8, Qwen3.8-Flash 7/8, DeepSeek-V4-Flash-Vision-Exp 8/8, under the logged tool). These observations motivate Phase II but are not confirmatory, for three disclosed reasons: the design was two nested contrasts that cannot separate protocol from builder main effects; the early runs lack per-attempt logs, so budget equality cannot be audited; and acceptance was an import/smoke check, not the counterfactual conformance gate of §4. Full condition details are in Appendix A.

Generation-null failures (M0). Re-running the original optimize loop with Qwen3.8-Flash and DeepSeek-V4-Flash-Vision-Exp builders produced 40 candidates byte-identical to the bare baseline source. Code identity alone does not imply identical stochastic executions. This run supplies no distinct-code population; it is reported as a distinct, more primitive failure mode — the pipeline produced no distinct population at all. The collapse claims above rest exclusively on the genuinely distinct-source day-1 population.

4 PHASE II: CONFIRMATORY GENERATION STUDY (DESIGN)

Phase II is the confirmatory study. Its protocol was frozen and committed before any confirmatory outcome was observed (one exception, disclosed as deviation D1 in the protocol changelog: an 18-item pipeline smoke test whose aggregate accuracies were observed and whose items were retained); deviations are logged in the protocol changelog (Appendix B). The frozen estimand, splits, instruments, and analysis plan are summarized here.

4.1 DESIGN

Arms. Two crossed factors: *strategy forcing* ($S=1$: the builder is assigned a named mechanism from a fixed set of eight; $S=0$: the builder invents an improvement with no mechanism named) and the *conformance gate* ($G=1$: the harness must pass the frozen conformance suite; $G=0$: neutral validity only). This yields four factorial arms, which we denote **II-A** (free/ungated), **II-B** (free/gated), **II-C** (forced/ungated), **II-D** (forced/gated). A fifth, pre-specified *secondary* arm **II-E** (open mechanism discovery: the builder emits a self-specified behavioral contract over generic trace primitives, gated on its own declaration) tests whether mechanism diversity requires author-supplied vocabulary at all; it is deliberately outside the factorial. These arm names are Phase-II-specific; Phase-I conditions in §3.5 are always referred to with their descriptive names.

Implemented treatment. The original gate maps both `hint_guard` and `format_guard` to a plain scenario that returns `NO_MECHANISM_declared`; admission requires `PASS`. Thus two of eight forced strategy slots cannot pass even if neutrally valid. We interpret G as *nontrivial-mechanism screening plus conformance verification*. The archived results do not identify pure verification, and matching population size cannot restore excluded types. Moreover, no `format_guard` candidate passed neutral validity in either forced arm (18 slots and 54 attempts per arm), before mechanism gating. An extraction audit identifies a possible upstream failure: embedded SQL fences can truncate otherwise valid Python (Appendix B).

Common attempt cap; unequal realized cost. Each slot has at most $R=3$ raw builder attempts (same model, temperature 0.7, 16,384 token cap); generation stops at the first accepted candidate.

The original protocol’s “exactly three” wording does not describe execution. Logged raw attempts total 269/255/273/324 for A/B/C/D, respectively, against a 432-attempt cap per arm; actual token costs are not fully recovered. Generation prompts and admission policies differ across arms: ungated arms take the first candidate passing neutral validity (parses, imports, returns SQL on a development question — mechanism-blind); gated arms take the first additionally passing the conformance suite. Slot failure is final; populations are never topped up, so generation reliability is part of the treatment. Extracted code files and attempt logs were saved, but complete builder messages were not archived by the generator. Generation reliability is reported separately from population quality via three rates: R_{artifact} (valid artifact), R_{contract} (valid, non-vacuous contract), R_{fidelity} (asserted properties verified, conditional on a valid contract).

Builders and seeds. Six builders from independent labs (GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview), three seeds each: 18 paired (builder $\times$ seed) cells. These were the strongest models available on the project’s single API gateway; no Anthropic- or OpenAI-family builder was accessible, so frontier capability-dependence is untested. Every cell enters the primary contrast regardless of admitted population size, including $K=0$ — excluding low-yield cells would condition on a post-treatment outcome.

Data isolation. A fail-closed contamination registry scans every project artifact for observed BIRD questions. Two databases touched during development are banned wholesale; the confirmatory evaluation uses **1,169 questions from nine databases never opened during development** (the registry’s defensible claim is no project-specific adaptive use, not pretraining-time isolation; the 18-item smoke contact is the disclosed exception), with a stratified 400-item core drawn before any outcome existed. Factorial secondary analyses run on the same core items for all four arms.

Judge. Primary: the official BIRD execution scorer (set equality on raw result rows, 30 s wall-clock timeout). Secondary: the Phase-I legacy judge, measured to be uniformly conservative (on Phase-I matrices it never credits an answer the official scorer rejects).

Instruments. The conformance suite drives each harness against scripted scenarios and asserts on the recorded execution trace, never on task performance, so gating cannot leak outcome supervision. Every mechanism is probed with *paired counterfactual variants* whose correct answer differs — including a variant where the first query *succeeds*, separating conditional repair from an unconditional second call — and answers are compared by exact normalized equality. The suite was calibrated before any Phase-II generation against three hand-written positive controls (all must pass) and two known-negative impostor harnesses (both must be rejected), plus the invariant that the bare baseline must violate every conditional property; the calibration is an executable gate re-run before each generation batch. Arm II-E additionally enforces non-vacuity (the contract must be violated by the bare baseline, checked by running it) and verifies only the properties actually asserted.

4.2 FROZEN ANALYSIS PLAN

The **primary estimand** is the paired difference in bare-inclusive oracle headroom, $H(\text{II-D}) - H(\text{II-A})$ over (builder $\times$ seed) cells, on the full 1,169-question set under the official judge — a continuous effect with a two-sided 95% CI and paired permutation test, never a pass/fail threshold. The Phase-I thresholds survive only as a descriptive checklist. **Confirmatory secondaries** (Holm-corrected, all four arms on the same core-400 items): gate main effect $[(B-A) + (D-C)]/2$, strategy main effect $[(C-A) + (D-B)]/2$, and their interaction $(D-C) - (B-A)$ — whether strategy differentiation requires execution fidelity to be realized. **Original plan and corrected inference:** the original primary code omitted within-database task resampling and sampled tasks separately across cells; the factorial code resampled cell summaries only. The versioned post-review analysis uses 10,000 shared database/within-database task draws across all arms and cells, keeping builders fixed and resampling generation seeds within builder. It does not add execution-repeat variance. Exact sign-flip reference tests are conditional on observed tasks and on cell-contrast sign symmetry (one-sided primary; two-sided secondaries with Holm adjustment). These are not randomized causal tests. $H(P)$ is *not* monotone in K — adding a harness that answers everything correctly drives headroom to zero — so matched- K comparison, not monotonicity, licenses cross-population claims. K -matched sensitivity, the full estimand definitions, and the robustness audits (dual judge,

Phase-II confirmatory design: database-level isolation, 2x2 factorial + open-mechanism arm, three-layer admission

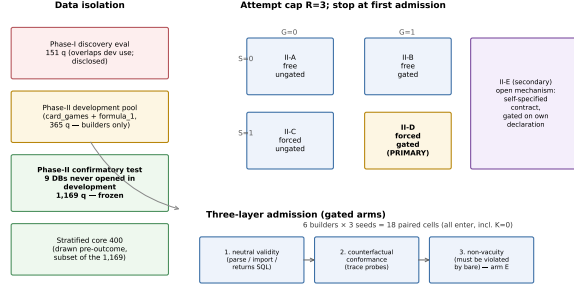


Figure 2: **Phase-II confirmatory design** (frozen protocol). Left: database-level contamination isolation (the two development databases are banned wholesale; evaluation uses nine databases, with the disclosed 18-item smoke exception). Middle: the 2x2 factorial (strategy forcing × conformance gate) with a common attempt cap (realized costs differ), plus the open-mechanism arm II-E. Right: the three-layer admission pipeline (neutral validity → counterfactual conformance → non-vacuity vs. bare).

Metric	II-A	II-D
Candidate count	5.78	3.83
Oracle accuracy (%)	76.14	75.70
Best-fixed accuracy (%)	67.58	67.30
Headroom (pp)	8.56	8.40
D—A headroom (pp)	−0.16	
Unadjusted bootstrap 95% CI	[−0.83, +0.87]	
One-sided sign-flip p	0.647	

Table 2: **Corrected primary analysis of the original intervention.** Means over 18 builder–seed cells on 1,169 questions. Bare is included in both the oracle and best-fixed comparator. The CI incorporates task and generation-seed variation, conditional on six fixed builders and the archived execution records. The sign-flip reference test conditions on the observed tasks and assumes symmetric cell contrasts.

cache-off replication, repeat stability) are specified in Appendix B together with the protocol deviation log.

5 PHASE II: RESULTS

5.1 CORRECTED REANALYSIS OF THE ORIGINAL GATE EXPERIMENT

The following tables report a post-review reanalysis of archived outcomes, not a new experiment with a repaired gate. Original analysis files remain available. All revised tables use full-precision outcomes from one canonical input order; the four-arm analysis reuses the same bare records on the same 400 tasks. The revised intervals use shared database and within-database task draws across all arms and cells, with generation seeds resampled within each of the six fixed builders.

The paired estimate is -0.16 pp with unadjusted 95% CI $[-0.83, +0.87]$ and one-sided sign-flip $p = 0.647$. We find no clear improvement; the interval does not establish equivalence. The full-set cell contrasts range from -4.36 to $+2.74$ pp. These are estimates for the implemented screening policy, which excludes some allowed strategies, rather than a pure verification intervention.

5.2 FACTORIAL EFFECTS AND ABSOLUTE PERFORMANCE

The gate estimate is -0.69 pp; its Holm-adjusted $p = 0.079$ does not meet the 0.05 threshold. This cannot distinguish useful verification from the gate’s structural exclusion of prompt transfor-

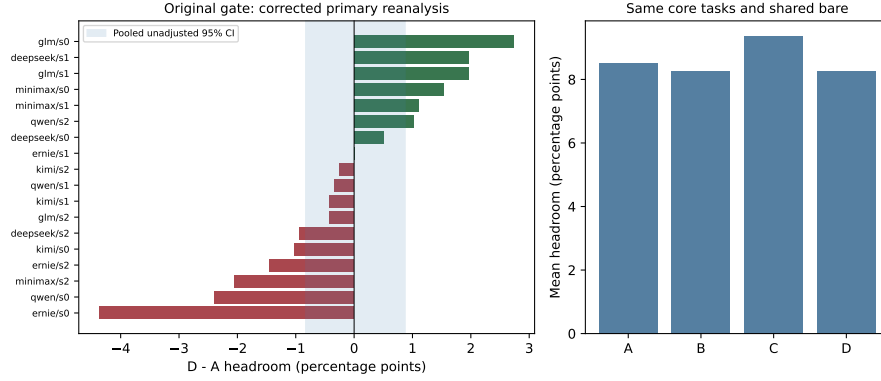


Figure 3: **Versioned corrected reanalysis.** Left: full-set D–A cell contrasts and the pooled unadjusted 95% interval. Right: four-arm headroom on the same core-400 tasks with a shared bare baseline. Both axes use percentage points. No significance stars are used; raw and adjusted p -values appear in Table 3.

Contrast	Effect (pp)	95% CI (pp)	Raw p	Holm p
Gate	−0.69	[−1.56, +0.20]	0.026	0.079
Strategy	+0.42	[−0.32, +1.51]	0.263	0.428
Interaction	−0.87	[−2.83, +0.65]	0.214	0.428

Table 3: **Corrected core-400 factorial reanalysis.** Gate is $[(B - A) + (D - C)]/2$, strategy is $[(C - A) + (D - B)]/2$, and interaction is $(D - C) - (B - A)$. CIs are unadjusted database/task/seed bootstrap intervals. Raw p -values use two-sided cell sign flips; Holm adjusts the three secondary tests. The two procedures address different sources of variation.

mations. The corrected fixed- K gate sensitivity is −0.31 pp (95% CI [−1.07, +0.40]). Both sides are matched to the smaller candidate count within each A/B and C/D cell; all candidate subsets are averaged exactly and bare is always retained (Appendix B). A change after subsampling does not identify a causal proportion mediated by population size. Moreover, oracle headroom is not monotone in population size: adding a strong fixed member can reduce it.

For the full primary set, the D–A oracle change is −0.44 pp and the best-fixed change is −0.28 pp. The best-fixed decrease offsets part of the oracle decrease; it does not explain away that decrease. The same-core four-arm decomposition is reported separately in Table 4.

5.3 EXECUTION VARIABILITY AND THE STABLE-COMPLEMENTARITY QUESTION

The archived R3 audit reports that 14.5% of tasks changed bare correctness at least once across three cache-off repeats. Canonical R2 replay finds 2391/28,000 same-harness verdict flips (8.54%). Retaining all 18 cells, including two with singleton A subsets, changes subset D–A from +0.42 to +0.81 pp; the change is +0.39 pp (95% CI [−0.95, +1.63]). R2 subset headroom excludes bare because R2 did not acquire a matched bare run. It is therefore distinct from the bare-inclusive primary estimand. The old 16-cell comparison is reconciled in Appendix B. These are instability measurements, not standard errors or a detectability floor for the mean treatment effect. They do not establish that every contrast is unchanged, or identify what fraction of between-harness disagreement is noise.

Let $p_{h,x}$ denote the probability that harness h answers task x correctly across independent executions. For a pool including bare, define

$$H_{\text{stable}} = \frac{1}{n} \sum_x \max_h p_{h,x} - \max_h \frac{1}{n} \sum_x p_{h,x}.$$

A same-code clone-only pool has identical $p_{h,x}$ across members, hence zero stable headroom despite potentially positive one-run headroom. Adding distinct bare can retain baseline–harness com-

Arm	K_{cand}	K_{total}	Bare (%)	Best (%)	Oracle (%)	H (pp)
A	5.78	6.78	63.25	66.00	74.51	8.51
B	6.11	7.11	63.25	66.12	74.39	8.26
C	5.61	6.61	63.25	65.64	75.01	9.38
D	3.83	4.83	63.25	65.65	73.90	8.25

Table 4: **Same-core headroom decomposition.** All arms use identical 400 tasks and identical bare outcomes. K_{cand} excludes bare and $K_{\text{total}} = K_{\text{cand}} + 1$; counts and accuracies are cell means. Headroom is oracle minus best-fixed accuracy, not realized routing gain.

plementarity. Independent repeats estimate a fitted policy’s value against its specified comparator. A discovery-selected fixed member may be suboptimal: beating it alone does not establish stable complementarity or unseen-task utility. The current data lack a balanced, cost-matched clone comparison, so stable added capability remains unestimated.

5.4 ROUTING AND EXPLORATORY SELECTION

A Phase-I leave-one-harness-out predictor failed to beat the best fixed harness in the DeepSeek population (-3.4 pp); the Qwen estimate was $+0.9$ pp with CI $[-5.2, +6.9]$. Database-stratified task-held-out controls reduced mean repair AUROC to 0.512, consistent with task memorization rather than reliable value prediction. These findings apply to the tested predictors and splits.

The canonical W1 reanalysis remains descriptive. At $k = 8$, with bare retained, diversity selection versus top-accuracy selection changes headroom by $+0.78$ pp: oracle changes by $+0.35$ pp and best-fixed by -0.42 pp. A fixed member chosen only on development data changes by -0.001 pp, with ties resolved consistently within each selected subset. These are different estimands, not a demonstrated routing gain. Appendix C reports absolute values and the limitations of this post-hoc, single-execution comparison.

6 DISCUSSION AND CONCLUSION

What the evidence establishes. Source diversity, executed mechanisms, and one-run correctness differences are distinct measurements. The Phase-I pattern did not recur in the revised Phase-II setting; multiple protocol changes prevent identifying the cause. The corrected analysis of the original gate shows no clear headroom improvement. Because that gate also excludes allowed prompt strategies, it does not identify the effect of verification alone.

What remains unidentified. Same-code reruns can disagree and create oracle gains. Outcome flip rates measure instability, not the standard error of a treatment contrast, and cannot define a numerical noise floor for that contrast. A matched-cost clone control, independent repeat validation, and validated calibration across mechanism types (Appendix G) are needed before attributing headroom to stable additional capability.

Limitations. The main comparison uses one text-to-SQL target and six fixed builders, with three generation seeds per builder. Phase I is discovery evidence with disclosed task overlap, a proxy judge, and a post-trace human control. The new statistical analysis is post-review and does not add independent executions. K-matched and R2 results now share the corrected data and resampling chain. W1/W3 now share the canonical data but remain descriptive; selection uncertainty and independent utility validation are outstanding. The completed one-seed MATH-500 free-arm probe (Appendix H) extends the observed outcome comparison to a second domain, but does not supply a clone control, a repaired-gate factorial, or an independent test of stable complementarity.

Generated code, observed outcome diversity, and reproducible additional capability should be evaluated separately. Our archived experiments expose failures at the first two levels and leave the third unresolved. Reporting absolute oracle accuracy, best-fixed accuracy, admission policy, execution cost, and repeat variability makes that boundary inspectable. Establishing practical value requires independently validated selection gains beyond same-code repeated sampling.

AI USE STATEMENT

This work is, by construction, an AI-assisted research project studying AI-generated artifacts, and we disclose the extent of that assistance precisely.

Required disclosure. We used generative AI tools for: (i) writing and modifying the experimental code (generation pipeline, evaluation collectors, statistical analysis scripts); (ii) running experimental workflows that made model API calls; (iii) analyzing experimental data and producing tables and figures; and (iv) drafting and editing manuscript text. The revised primary and factorial tables and result figure are generated from a shared, versioned reanalysis artifact. Historical exploratory analyses retain their original provenance and stated limitations. The post-review corrections are recorded separately from the original protocol. Verification records specify the scope of automated tests, development controls, and review for each artifact. Post-review revisions also used same-model subagent reviews; these are mechanical checks, not independent cross-model scientific validation. Development controls and local runtime tests do not establish independent gate calibration or provider-level execution independence.

Research ideas and direction. The core research questions, the causal design decisions (estimand, contrasts, exclusion rules), the interpretation standards, and proposed go/no-go criteria were developed with AI assistance. The human authors set the overall research direction and requested the review-driven revision; AI tools also proposed analysis and experiment designs, identified counterexamples, and implemented revisions. Final scientific decisions and approval of the submission remain the authors’ responsibility. This disclosure does not claim that every AI-proposed decision has already received individual author approval.

Responsibility. We take responsibility for the final content of this work, including text, claims, and artifacts produced with the aid of generative AI. The factual statements in this section are provided for author verification; final responsibility for their accuracy rests with the authors.

REFERENCES

- Mengzhuo Chen, Junjie Wang, Zhe Liu, Yawen Wang, Haiming Zheng, and Qing Wang. From failed trajectories to reliable LLM agents: Diagnosing and repairing harness flaws. *arXiv preprint arXiv:2606.06324*, 2026.
- Yong-eun Cho. It’s not the capability: Harness sensitivity is non-monotone across LLM agent tiers. *arXiv preprint arXiv:2605.26731*, 2026.
- Tianyu Fan and Chao Huang. HELIX: Model-harness co-evolution for recursive self-improvement. *arXiv preprint arXiv:2608.13951*, 2026.
- Akshat Gupta, Jermaine Lei, Alexander Lu, Gopala Anumanchipalli, and Leshem Choshen. Automated discovery has no universally superior harness. *arXiv preprint arXiv:2607.18235*, 2026.
- Minho Kim. When should LLMs search? counterfactual supervision for search routing. *arXiv preprint arXiv:2607.05752*, 2026.
- Lars Kotthoff. Algorithm selection for combinatorial search problems: A survey. *AI Magazine*, 35(2):48–60, 2014.
- Jinyang Li, Binyuan Hui, Ge Qu, Jiayi Yang, Binhua Li, Bowen Li, Bailin Wang, Bowen Qin, Ruiying Geng, Nan Huo, et al. Can LLM already serve as a database interface? a BIRD’s eye view. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Lightman et al. let’s verify step by step appendix: Math-500 subset. *arXiv preprint arXiv:2305.20050*, 2023.
- Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, and Hang Yan. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. *arXiv preprint arXiv:2604.25850*, 2026.

- Xinchen Liu, Hang Zhou, Yingjie Zong, Yuchuan Tian, Liuyang Song, and Shuo Zhang. Agentic routing: The harness-native data flywheel. *arXiv preprint arXiv:2607.11399*, 2026.
- Xiaotian Luo, Dizhan Xue, Fengxingyu Wang, Chuanrui Hu, and Yafeng Deng. HarnessBank: Semantic gene-bank search with gated verification for agent-harness self-evolution. *arXiv preprint arXiv:2607.13683*, 2026.
- Atsuyuki Miyai, Kiyoharu Aizawa, and Toshihiko Yamasaki. Task-CoEvolve: Efficient harness optimization via adaptive validation task selection. *arXiv preprint arXiv:2608.20169*, 2026.
- Johannes Moll, Jean-Philippe Corbeil, Jiazhen Pan, Martin Hadamitzky, Daniel Rueckert, and Lisa Adams. GRASP: Gated regression-aware skill proposer for self-improving LLM agents. *arXiv preprint arXiv:2605.29668*, 2026.
- Jun Nie, Yonggang Zhang, Jun Song, Qianshu Cai, Dahai Yu, Xinmei Tian, Bo Han, Yike Guo, et al. TTHE: Test-time harness evolution. *arXiv preprint arXiv:2607.08124*, 2026.
- John R. Rice. *The Algorithm Selection Problem*. Elsevier, 1976.
- Ruhan Wang, Yucheng Shi, Zongxia Li, Zhongzhi Li, Yue Yu, and Junyao Yang. Harness handbook: Making evolving agent harnesses readable, navigable, and editable. *arXiv preprint arXiv:2607.13285*, 2026.
- Jinghan Xu, Yikai Zhang, Aili Chen, Weiyuan Li, Jiaqing Liang, and Deqing Yang. Verify smarter, evolve further: Efficient harness evolution through behavior-aware verification. *arXiv preprint arXiv:2608.27311*, 2026.
- Lin Xu, Frank Hutter, Holger H. Hoos, and Kevin Leyton-Brown. SATzilla: Portfolio-based algorithm selection for SAT. *Journal of Artificial Intelligence Research*, 32:565–606, 2008.
- Guibin Zhang, Leo Lu, Fangzhou Xie, Kang Zhu, Junhao Wang, and Zhifei Xie. JIT-Agent: Scaling harness intelligence via just-in-time harness evolution. *arXiv preprint arXiv:2608.25593*, 2026a.
- Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, and Yiqun Zhang. Self-harness: Harnesses that improve themselves. *arXiv preprint arXiv:2606.09498*, 2026b.
- Heng Zhou, Zelin Tan, Zhemeng Zhang, Yutao Fan, Yibing Lin, and Li Kang. Select-then-solve: Paradigm routing as inference-time optimization for LLM agents. *arXiv preprint arXiv:2604.06753*, 2026.

A PHASE-I PROVENANCE AND CONDITIONS (DETAIL)

Discovery status. Phase I (Sections 3 and 5.4) is a discovery study. 24 of its 151 evaluation items overlapped day-1 protocol-development measurements; the human control was designed after inspecting day-1 traces (its observed headroom is descriptive, not a blind estimate of stable complementarity); the repository history was initialized retrospectively, and the decision-log changelog is the authoritative ordering record. Phase-I admission thresholds (union repair $\geq 15\%$, headroom ≥ 5 pp, ≥ 3 pairwise-distinct non-empty fix sets) were frozen in the internal decision log before the 151-item evaluation; the v6 $\rightarrow$ v7 threshold change (20% $\rightarrow$ 15%) is disclosed, and both readings admit the same populations. This is an internal pre-evaluation freeze, not external preregistration.

Data hygiene. The bare baseline flipped 3 of 60 items between the two day-1 runs (API nondeterminism at temperature 0), so 60-item columns are conditioned on slightly different bare-error sets. Phase-I judging used a set-of-rows-equality proxy that we later measured to be uniformly conservative relative to the official BIRD scorer (on the two Phase-I matrices with logged final SQL, it never credits an answer the official scorer rejects; agreement 0.97–0.99).

Phase-I populations. Seed baselines: `bare` (single call) and `react` (execution feedback loop, shipped with TTHE). *Old proposer protocol*: six candidates selected from TTHE’s optimize-loop output (GLM-5.3-Flash builder), selected for behavioral diversity using only the 36 questions disjoint from the evaluation set, by a rule frozen before evaluation. *Strategy-forced free-form*: GLM-5.3-Flash, Qwen3.8-Flash, and DeepSeek-V4-Flash-Vision-Exp builders, eight assigned mechanisms each; the GLM population is the logged re-run ($K=3$; the original 09-01 run recorded zero accepted outputs but its tool was not preserved and the observation is unaudited). *Harness IR*: six harnesses compiled deterministically from declarative mechanism specifications ($\text{context} \in \{\text{plain}, \text{schema_link}, \text{hint_guard}, \text{link+hint}\}$), candidate generation with temperature, per-candidate execution, repair rounds, selection rule), so a declared mechanism is present in the control flow by construction. *Human positive control*: the four hand-written harnesses of §3.4.

Generation-provenance gaps. The 09-01 free-form populations were produced in-session; their generation tool was not preserved, and per-attempt logs do not exist for them — so budget equality cannot be audited for those runs, and we do not claim it. Their acceptance was an import/smoke check at evaluation time, *not* the Phase-II counterfactual conformance gate. The tool of record from 2026-09-03 (`builder_generate3.py`) logs every attempt (3 retries, 16,384-token budget, per-attempt stage and finish reason); the completed per-strategy grids and the builder-acceptance ordering (3/8, 7/8, 8/8) come from this logged tool and are post-hoc with respect to the frozen admission decisions.

B PHASE-II PROTOCOL SUMMARY AND DEVIATION LOG

The authoritative protocol (estimands, splits, instruments, exclusion rules, and the historical deviation changelog) is `PHASE2_PROTOCOL.md` in the code release, frozen at the commit recorded in `PROTOCOL_FREEZE.txt`. Summary: 2×2 factorial plus open-mechanism arm; six builders $\times$ three seeds; at most $R=3$ attempts per slot, stopping at first admission, with no top-up; contamination registry (fail-closed, two development databases banned wholesale); 1,169-question confirmatory set from nine databases never opened during development plus a pre-drawn 400-item core; official BIRD judge with enforced 30 s timeout; response cache keyed on model identity with cross-process locking (deviation D13 documents and fixes a cache concurrency bug found during collection); primary contrast D–A on paired (builder $\times$ seed) cells with hierarchical bootstrap and permutation inference; Holm-corrected factorial secondaries on the common core; K-matched sensitivity only. Deviations D1–D13 include the disclosed 18-item pipeline smoke test, instrument-calibration fixes made before any confirmatory outcome was observed, the credential-handling incident and its remediation, and the artifact-divergence quarantine — each with its rationale and date in the changelog. The post-review corrections are recorded in `review-stage/REVISION_20260910.md`; revised results do not replace the historical freeze. In particular, the implemented gate structurally rejects two allowed prompt-level strategies and realized generation costs differ across arms. Corrected tables use shared baseline records and database/task resampling, as described in the main text.

Upstream code-extraction audit. Across the 36 forced C/D cells, all 108 `format_guard` attempts failed neutral validity: 71 logs report unterminated Python strings, 36 report no Python fence, and one reports a provider timeout. The archived generator’s non-greedy Python-fence extractor terminates at the first subsequent triple backtick, including one inside a Python string. Three synthetic, syntactically valid files containing SQL fences in an ordinary string, docstring, or regex become invalid prefixes; the extractor nevertheless sets its truncation flag to false. This extractor is unchanged from the review-locked commit, but the historical attempt logs do not bind its execution-time source hash. Full builder messages were not saved by this generator. Thus the failure pattern is compatible with extraction damage, not proof that all historical failures had this cause or that repaired candidates would pass admission. We found 74 saved extracted files, all syntactically invalid; three coexist with no-fence logs and are retained as provenance conflicts, not attributed to those attempts. These findings place a generation interface failure alongside the gate’s explicit strategy exclusions.

Corrected K-matched and R2 replays. Both replays use the same manifest-ordered canonical loader and shared database/within-database task draws as the corrected primary analysis, with seeds

Sensitivity estimand	Estimate (pp)	95% CI (pp)
K-matched B–A headroom	−0.55	[−1.53, +0.63]
K-matched D–C headroom	−0.08	[−1.26, +0.80]
K-matched gate mean	−0.31	[−1.07, +0.40]
R2 subset D–A (cached)	+0.42	[−0.61, +1.48]
R2 subset D–A (cache-off)	+0.81	[−0.35, +1.84]
R2 change in subset D–A	+0.39	[−0.95, +1.63]

Table 5: Corrected sensitivities on core-400, 18 cells. CIs are unadjusted. K-matched comparisons always retain bare; R2 comparisons exclude bare and are conditional on the observed acquisition groups.

Arm	Candidate-task records	Logged solver calls	Calls/task/population
A	41,600	67,258	9.34
B	44,000	64,399	8.94
C	40,400	75,604	10.50
D	27,600	52,043	7.23

Table 6: Descriptive canonical cost audit on core-400, 18 cells per arm. Logged logical calls are not billable requests or token counts. The last column averages candidate-population calls per task over cells.

resampled within fixed builders (10,000 replicates). For K-matching, each A/B and C/D pair uses the smaller candidate count; every uniform candidate subset is enumerated exactly. Bare stays in every subset, and best-fixed is recomputed inside each subset on every task draw. This is a size sensitivity analysis, not a causal mediation decomposition.

R2 acquired 70 harnesses on 400 tasks, selecting the first two callable admitted candidates per A/D cell where available. The two cache-off acquisition files retain separate identities even though both stored `repeat=0`; source hashes are checked against the cached runs. No matched bare was acquired, so R2 headroom excludes bare in both conditions. The corrected analysis retains singleton cells (their subset headroom is zero), yielding all 18 paired cells. There are 2391/28,000 verdict flips against cached outcomes (8.54%) and 1245/14,400 flips between the two D cache-off passes (8.65%). For reconciliation only, restricting to the 16 cells with at least two members in both arms gives D–A −0.11 pp cached and +0.36 pp cache-off. This explains the difference from the full-cell estimates; neither analysis identifies future execution variance, the effect of caching alone, or a stable-complementarity advantage over clones. Acquisition time and caching condition are not isolated.

Recorded cost and missing billing data. The same canonical core records contain 259,304 logged logical solver calls across 153,600 candidate–task records (Table 6). These calls include cache-served requests and are not provider-billed API requests. None of the inspected records has token usage fields; we do not impute token or monetary expenditure from call counts. Actual generation attempts, admitted candidate counts, and these execution counts therefore remain separate measurements. The population cost mean gives each builder–seed cell equal weight and excludes bare; zero-candidate populations would contribute zero solver calls.

C EXPLORATORY FINGERPRINT AND SELECTION ANALYSES

Both analyses are post-hoc on the archived confirmatory cells (18 builder×seed cells, 384 admitted candidates, core-400 outcomes). The versioned `experiment/revision/` scripts now use the same canonical loader, frozen task list, membership and source checks as the main reanalysis. They remain descriptive: these tasks had already been analyzed, and this is not independent transfer validation.

Rich fingerprints. To isolate the loader change, the W3 reanalysis retains the legacy lossy SQL text transform: semicolon deletion, backtick replacement, lowercasing even segments after splitting

k	Selection	Oracle (%)	Best (%)	H (pp)	Dev-fixed (%)
4	top-acc	73.98	66.64	7.34	65.10
4	random	73.90	66.03	7.87	64.66
4	div-only	74.22	66.18	8.04	65.13
8	top-acc	76.03	67.11	8.93	65.10
8	random	76.15	66.76	9.39	64.95
8	div-only	76.39	66.68	9.70	65.10

Table 7: Canonical descriptive W1 reanalysis. Every pool contains k selected candidates plus the common bare. Means over 18 cells and 100 overlapping database splits; no confidence intervals. Dev-fixed is chosen only on development data; Best uses held-out outcomes.

on single quotes, and whitespace joining. This is not a SQL parser or semantic normalization; it can alter literals and quoted identifiers. Among 364,428 pair-task observations with two identical, nonempty transformed SQL strings, 0 have different verdicts. This is an observed consistency check, not a property guaranteed by construction.

Pair-level SQL dissimilarity averages (1-token-Jaccard) only over verdict-agreeing tasks with two nonempty SQL strings. Its Spearman correlation with overall outcome disagreement is 0.597 across 4,386 within-cell pairs and 0.588 across 1,010 within-arm pairs. Pairs share candidates and tasks, and the SQL metric conditions on verdict agreement; these correlations have no independence-based significance or causal interpretation.

In R2’s 70-harness first cache-off acquisition, call counts change on 506/28000 comparisons and verdicts on 2391/28000. Transformed SQL changes on 19569/27936 comparisons (70.05%); 64 comparisons lack two nonempty SQL strings and are excluded only from the SQL denominator. The mean token Jaccard among changed SQL pairs is 0.619. These channel-specific rates do not define an effect-size noise floor. Canonical logical-call costs are reported separately in Table 6.

Diversity-aware selection. Each cell pools all admitted candidates across arms. The reanalysis retains the legacy seed, 100 random database-half splits, 200 random subset draws per split and cell, and candidate-selection rules. It records a new deterministic schedule: the historical set-derived cell order was not archived, so identical historical splits and draws cannot be recovered from the seed alone. Selection uses development databases; evaluation uses the complementary databases. The overlapping splits are not 1,800 independent experiments. Methods are top- k development accuracy, uniform random subsets, greedy development oracle coverage, and greedy mean accuracy plus λ times oracle coverage for $\lambda \in \{0.25, 0.5, 1, 2\}$. All candidate choices are frozen within a split before its evaluation.

Table 7 retains the common AD bare in every evaluated subset; k counts candidates only. The JSON also records candidate-only values to distinguish the historical estimand. Oracle and best-fixed use held-out outcomes as descriptive upper bounds. Dev-fixed instead chooses one member using development accuracy, with ascending member-name ties and bare first. Deterministic methods already retain a globally dev-best candidate; differences in this comparator may arise from which tied maxima survive, not improved maximum development accuracy.

At $k = 8$, div-only versus top-acc changes oracle by +0.35 pp, best-fixed by −0.42 pp, and headroom by +0.78 pp. Dev-fixed changes by −0.001 pp. These are distinct quantities, and none is an independently validated routing gain. We withdraw the legacy cell-only confidence intervals; the current table is descriptive while inference covering database/task sampling and re-selection remains outstanding. The earlier normalized-headroom ratio is retained only in the historical artifact and is not carried into the revised claims.

Re-selection does not establish interval coverage. An exact counterexample rules out a universal 95% coverage guarantee for our proposed positive-weight re-selection percentile interval. Let nine independent databases each contain one Bernoulli(p) task state. Sixteen fixed candidates comprise eight state-0 specialists followed by eight state-1 specialists; bare is always wrong. Apply the canonical $k = 8$ rules to four development and five held-out databases. With the recorded tie rules, the expected div-only minus top-acc headroom statistic is $(1 - p^4) \mathbb{E}[\min(B, 5 - B)/5] > 0$, where

$B \sim \text{Binomial}(5, p)$. On the event that all nine states coincide, every positive reweighting produces zero headroom for both methods, so the percentile interval is $[0, 0]$. Coverage is therefore at most $1 - p^9 - (1 - p)^9$, or 61.26% at $p = 0.1$. Averaging identical cells or overlapping fixed splits does not remove this event. This bound concerns the stated construction and interval, not BIRD-specific coverage or the impossibility of other inference methods.

D PHASE-I ADDITIONAL RESULTS

Per-harness accuracies, per-fold LOHO tables, bootstrap intervals, fix-set audits, K-subsampling curves, the task-held-out control, and the second-target replication (Qwen3.8-Flash as frozen target: collapse replicates; the strategy-forced population retains headroom; the prompt-side human control does not transfer wholesale) are included in the anonymous code release with their generating scripts.

E MODELS AND VERSIONS

Table 8 lists every model used in this paper with its exact version string as sent to the API. All models were accessed through a single commercial OpenAI-compatible API aggregator (the same provider for every role); the BIRD data-collection window is 2026-08-30 through 2026-09-08. The frozen target is always called at temperature 0. Phase-II harness generation uses temperature 0.7 with a 16,384-token budget; Phase-I generation settings are recorded per tool in the code release. Phase I used a proxy execution judge; Phase II uses the official BIRD scorer (30 s timeout), with the legacy judge as a secondary audit. The MATH-500 probe uses the answer comparator in Appendix H; no LLM judge produces reported accuracy.

Role	Model (exact version string)	Used in
Frozen target solver	GLM-5.3-Flash (temperature 0)	Phases I–II
Second frozen target (replication)	Qwen3.8-Flash	Phase I
Phase-I builders	GLM-5.3-Flash, Qwen3.8-Flash, DeepSeek-V4-Flash-Vision-Exp	\$3.5
Phase-II builders	GLM-5.3, Qwen3.8-Max, DeepSeek-V4-Pro, Kimi-K3, MiniMax-M3, ERNIE-5.0-Thinking-Preview	Phase II
Judge	proxy / official BIRD execution scorer; MATH answer comparator	see text

Table 8: Complete model inventory. The target is held frozen within each phase; Phase-II builders come from six independent labs.

F WORKED EXAMPLES FROM THE PHASE-I TRACE AUDIT

To make the failure taxonomy of §3.3 concrete, this section walks one BIRD task through the audited harnesses. Artifacts are verbatim from the day-1 trace audit (5 harnesses $\times$ 10 tasks); its logger caps stored SQL excerpts at 120 characters, so truncations are marked *[trunc.]*. Full harness sources and traces are in the code release.

The task. BIRD dev-set question 352 (`card_games` database; question text verbatim, the typo is the dataset’s):

```
Calculate the percentage of the cards availabe in Chinese
Simplified.
```

Its BIRD hint:

```
'Chinese Simplified' is the language; percentage =
Divide(Sum(id where language = 'Chinese Simplified'),
Count(id)) *100
```

Gold SQL (executing the two on the released database yields 8.7734 and 8.7728 respectively — the official scorer separates them):

```
SELECT CAST(SUM(CASE WHEN T2.language = 'Chinese Simplified'
THEN 1 ELSE 0 END) AS REAL) * 100 / COUNT(T1.id) FROM cards
AS T1 INNER JOIN foreign_data AS T2 ON T1.uuid = T2.uuid
```

The bare harness answers:

```
SELECT SUM(CASE WHEN language = 'Chinese Simplified'
THEN 1 ELSE 0 END) * 100.0 / COUNT(id) AS percentage FROM
foreign_data;
```

marked wrong by the official scorer: it counts directly over `foreign_data`, whereas the gold query counts after joining to `cards`; the hint does not spell out that join.

react (T2: mechanism present but untriggered). `react`’s control flow does execute SQL and retries on database errors. But its first call is byte-identical to `bare`’s prompt, and here the generated SQL executes without error, so the error-triggered retry never fires and the final SQL equals `bare`’s. Across all 10 audited tasks the pattern held: SQL executions present, final SQL byte-identical to `bare`’s on 10/10 — the feedback loop is real code whose repair branch did not activate on these tasks. Controlled failure injection is needed to test whether feedback changes the output.

cand.bird.g1.b0r1.g0 (code-level diversity, behavioral identity). This candidate is a genuinely different program: it wraps the single coder call in a daemon-thread watchdog and fires one hedged duplicate re-ask if the call stalls past 40 s. The mechanism exists in the control flow — but its trigger (a stalled API call) never occurred in the audit: zero SQL executions, and the final SQL equaled `bare`’s on all 10 audited tasks. Code difference without behavioral difference survives even when the added mechanism is real: it simply never fires.

cand.bird.g1.b1r1.g1 (T3: a broken multi-turn path). The one audited candidate implementing genuine execute-inspect-repair returns the model’s *prose* as final SQL. On a task `bare` answered correctly (“State the alternative languages available for card named Annul numbered 29.”), its logged final SQL begins, verbatim:

```
with "only a single read-only SELECT is allowed". The
previous "query" was actually just prose text (the model
wrote rea[trunc.]
```

The repair re-prompt invites the failure: asked to fix a query that “was actually just prose”, the frozen target responds with more prose reasoning, no SQL block is found, and the reasoning text is submitted as the query. The item flips from `bare-correct` to `harness-wrong`.

G INDEPENDENT CONSTRUCTION OF INSTRUMENT CONTROLS

We tested the post-review Gate-v2 development instrument with 32 newly authored controls, separate from the controls used to develop its probes. The instrument was fixed before a separate model received the public contracts and callable interface, without its implementation or earlier controls. The author supplied two intended conforming and two intended nonconforming implementations for each of the eight mechanism types. Control source, author-reference labels, instrument, measurement code, and execution order were hashed before the first evaluation. No benchmark or provider generation was run, and this first measurement is retained unchanged.

The nine review-required cases were retained. A third model received the full control source and traces without author-reference labels or automatic verdicts, for source-contract review of all 32 controls and semantic adjudication of the four designated mechanism types. Independent source review supported conformance for one decomposition control and two error-classification controls that the instrument rejected. The decomposition stub returned the payload of the first plan step whose text occurred in a request; retaining the earlier step in the context of the next request therefore

Declared mechanism	Structural pass	Structural fail	Review required
Repair	2	2	0
Execution-result vote	2	2	0
Schema linking	0	4	0
Hint guard	0	0	4
Two views	0	3	1
Decomposition	1	3	0
Error classification	0	4	0
Format guard	0	0	4
Total	5	18	9

Table 9: First evaluation of 32 independently authored controls by the fixed development instrument. No case had an execution failure. These are instrument states, not correctness labels or population error rates; the author’s intended positive/negative labels require separate scrutiny.

retrieved the earlier payload again. The classification check removed the exact error text but still counted an ordinary `Schema :` heading as a category, making the extracted category set disagree with the expected error class. These are instrument failures on these controls, not observed failures of a target model. Three author-intended positive controls were disputed: two schema-linking controls retained an unrelated table, and one two-view control batched its requests and mishandled an empty result. Their original labels and the review disagreements were retained separately. Among the 29 controls whose author labels and source review agreed, the instrument rejected 3 of 13 conforming controls; all 16 nonconforming controls were rejected after the designated semantic review. The instrument therefore did not pass this finite control check. These counts do not estimate error rates in a generated population. Semantic adjudication of the designated 16 controls yielded 7 conforming, 8 nonconforming, and 1 indeterminate case. The indeterminate case already had a structural failure, which was retained without resolving its semantics. This small, deliberately balanced control set is not a random sample of generated harnesses. Passing it would not validate the production interface or replace the common-raw-pool comparison of admission policies. The source, frozen inputs, full traces, and adjudications are retained in the accompanying revision artifacts.

Revised instrument and a new independent control set. Gate-v3 corrected decomposition request attribution and treated category keywords and shared request templates as review clues rather than automatic rejections. We fixed this version before fresh model contexts authored and reviewed a new set of 32 controls, again two intended positive and two intended negative controls per mechanism. The review packet withheld author labels, automatic verdicts, and instrument-derived attribution metadata. All 32 controls received source review; the 20 controls in the four prompt/formulation groups plus decomposition also received semantic or data-flow review. Structural outcomes were 8 passes, 10 failures, and 14 review-required cases, with no execution failures. Following one recorded evidence clarification, source review classified 15 controls as conforming and 17 as nonconforming; semantic review classified 8 as conforming, 11 as nonconforming, and 1 as indeterminate.

The indeterminate classification control requested an intermediate error category, but the synthetic interface returned SQL text. Its fallback then selected a semantics repair action. This establishes a response-type mismatch in the fixture, not a real solver’s classification error. A second classification control was reference-disputed: its source mapped the supplied schema-failure message to a semantics action. Among the 31 undisputed references, 14 of 15 conforming controls were accepted and one remained unresolved; all 16 nonconforming controls were rejected. The prespecified finite check therefore remained unmet because of the unresolved case and insufficient agreed positive coverage for error classification. One voting control’s conformance applied only to result arbitration; it did not establish task-conditioned candidate generation. These finite mechanism checks do not validate complete harness operation or replace the planned common-pool policy comparison.

Runtime response contracts. We subsequently tested seven archived ungated, strategy-forced candidates, selecting the first available filename per strategy rather than by performance; no format-guard candidate was available (Appendix B). The 27 synthetic scenarios exposed both candidate and instrument limitations. The two-view candidate skipped its second execution when the first result was nonempty, contrary to its original “execute both” declaration. In contrast, the schema-linking

candidate requested JSON but received plain text, and the full plan in decomposition requests prevented the frozen probe from uniquely attributing the current target. We retained those observations and tested source-assigned response profiles in 11 further development scenarios. Dedicated sub-question fields identified the current target, executable SQL payloads reached the final assembly, and four errors produced by an in-memory SQLite database elicited the corresponding syntax or schema repair instructions. Schema-linking JSON changed the explicit table/column restrictions in the SQL-writing request, although the full schema remained present. Retaining schema context therefore cannot by itself establish that the linked subset was ignored. These exposed-candidate checks distinguish response-interface failures from observable candidate deviations; they do not replace independent calibration or provide revised benchmark outcomes.

Unified interface and premeasurement reference review. Gate-v4 combined the source-assigned response interfaces with the five unchanged Gate-v3 mechanism probes. A separately authored set of 32 controls underwent blinded source review before execution. The author was familiar with earlier control failures; schema-response and declared voting-parameter mismatches found in this source review were corrected, with the initial versions retained. The revised references had two agreed conforming and two agreed nonconforming controls per mechanism. After freezing their source, profiles, and dependencies, one measurement covered 124 scenarios without execution exceptions. Structural results were 4 passes, 6 failures, and 22 review-required cases.

A new review context received all source and actual traces without reference labels, source-review decisions, or automatic verdicts. Its first judgments were 18 conforming and 14 nonconforming. Retaining structural failures in the prespecified hybrid rule accepted all 16 conforming references but also one nonconforming hint control, so the finite zero-error criterion was not met. That control correctly restated existing hints but invented a mandatory sort order when no hint was supplied. A recorded, targeted evidence clarification corrected this oversight and a two-view judgment that had treated two samples from one request as independent formulations. The latter control was already rejected structurally. The clarified decisions agreed with all 32 references, but this follow-up does not replace the first blinded result or constitute a new independent calibration pass. No instrument rerun, provider generation, or benchmark evaluation was performed in this check.

H CROSS-DOMAIN REPLICATION: MATH-500 MINI-AUDIT

To test whether the Phase-II clean-protocol finding (free ungated generation produces distinct one-run correctness vectors) is text-to-SQL-specific, we replicated the free-arm protocol on competition math.

Protocol. Six builders (the same models as Phase II), one seed, eight slots each, an $R=3$ raw-attempt cap per slot, neutral validity gate (mechanism-blind: parse, import, interface, non-empty answer on two dev questions). Frozen target GLM-5.3-Flash (same as the SQL study), temperature 0. MATH-500 (Lightman et al., 2023): 500 tasks frozen before generation (100 dev / 400 eval, seed 20260908). Judge: numeric equality where both sides parse as numbers (including a/b , $\frac{a}{b}$, decimals), else normalized LaTeX string equality (units, base subscripts, $^\circ$ stripped; $\text{\texttt{\text{X}}}$ content preserved; $\text{\texttt{pmatrix}} \rightarrow \text{tuple}$). Judge self-consistency: 400/400 gold answers judge correct against themselves. bare accuracy 0.938 under the matched answer-format instruction. Yield: 35/48 slots admitted (R_{artifact} 0.10–1.00; the two reasoning-heavy builders, GLM-5.3 and Qwen3.8-Max, exhausted the 32k-token generation budget on reasoning before emitting code in most slots — reported as-is). This is a replication probe, not a factorial: no strategy forcing, no conformance gate.

Results (35 harnesses, 400 eval tasks). Mean pairwise outcome disagreement 10.6% (max 33.0%), K_{eff} 33/35 (0.94), outcome-identical pairs 0.7% — no trace of Phase-I-style collapse. Union repair 76.0% of bare errors, union harm 0.8%, oracle headroom over bare 4.75 pp. Yet the best fixed member is *bare itself* (0.9375; generated harnesses 0.68–0.87): observed oracle coverage exceeds bare while every generated member is worse individually. Repeated-execution and same-code clone controls were not collected, so this does not establish stable complementarity or deployable utility.

Two failure-side observations. First, the clean-protocol diversity result replicates in a second domain with a different output space. Second, a quarantined pilot round (disclosed in `PROTOCOL_CHANGELOG.md`, entry G1) supplies an exploratory comparison: when the same six builders were (mistakenly) given a GSM8K-style task description whose numeric answer format did not match MATH-500, the generated population remained behaviorally diverse (16.2% disagreement, K_{eff} 19/19) while its union repair collapsed to 18% — and code-level inspection of the catastrophic members reproduced the Phase-I trace taxonomy verbatim (a generate-then-verify harness whose verifier rejected nearly every answer and whose fallback returned the stale loop variable or “0”). Diversity without utility, induced by a prompt-side protocol error invisible to code-level checks: the measurement mandate, not any code-level audit, is what detects it.

Scope. One seed, one target, no factorial, exploratory; the Phase-I collapse-inducing SQL protocol was not re-run in the math domain, so this replication speaks to the clean-protocol result only, not to the boundary of collapse.